MNIST Handwritten Digit Recognition

Using Convolutional Neural Networks (CNN) | TensorFlow / Keras

Final Project Report • March 2026

Dataset	Framework	Best Model	Test Accuracy
MNIST	**TensorFlow/Keras**	**CNN (SGD lr=0.01)**	**99.48%**

1. Executive Summary & Problem Understanding

Dataset Description

The MNIST (Modified National Institute of Standards and Technology) dataset is the benchmark standard for handwritten digit recognition. It contains 70,000 grayscale images (60,000 training, 10,000 test) of handwritten digits 0-9, each of size 28x28 pixels with pixel values in the range [0, 255]. The dataset is perfectly balanced (~9.7-11.2% per class) with zero missing values and no duplicate images.

Problem Statement

Build an end-to-end machine learning pipeline to classify handwritten digit images (0-9) with maximum accuracy. The pipeline must encompass data preprocessing, exploratory analysis, classical ML baselines, ANN experimentation, and a final CNN model — all implemented using user-defined functions and sklearn-compatible Pipelines.

Objectives

- Perform thorough EDA and extract minimum 5 meaningful insights from the dataset.
- Engineer at least 2 new features (mean_intensity, pixel_std) and apply feature selection via PCA.
- Train and compare 4 classical ML classifiers using sklearn Pipeline architecture.
- Build an ANN with minimum 6 hidden layers; experiment with Adam and SGD optimizers.
- Train a CNN and compare all models in a master performance table.
- Identify the best model and provide real-world business interpretation.

Key Findings Summary

99.48%	**99.48%**	**100.0%**	**10 / 10**	Synthetic Pred

The CNN model with SGD optimizer (lr=0.01, momentum=0.9) achieved the highest performance across all metrics. Classical ML models (SVM best at 94.15% F1), ANN models (SGD best at 98.30% F1), and CNN (99.48% F1) demonstrate a clear performance hierarchy favouring spatial feature learning via convolution.

Final Model Recommendation

Recommended Model: CNN with SGD optimizer (lr=0.01). The CNN architecture extracts hierarchical spatial features — edges, curves, and strokes — that are inherently present in handwritten digit images. With BatchNormalization, Dropout regularisation, and ReduceLROnPlateau scheduling, it achieves 99.48% accuracy while remaining robust against overfitting. This model is recommended for any production OCR or digit-recognition deployment.

2. Data Cleaning & Preprocessing

Dataset Structure

Attribute	Training Set	Test Set
Total Samples	60,000	10,000
Image Shape	28 x 28 pixels	28 x 28 pixels
Channels	1 (Grayscale)	1 (Grayscale)
Pixel Range (raw)	0 - 255 (uint8)	0 - 255 (uint8)
Classes	10 (digits 0-9)	10 (digits 0-9)
Missing Values	0 (0.00%)	0 (0.00%)
Duplicate Images	0	0

Preprocessing Pipeline (sklearn Pipeline)

All preprocessing steps are encapsulated within a reusable sklearn Pipeline. Each step is implemented as a user-defined function wrapped in FunctionTransformer, ensuring reproducibility and preventing data leakage.

Step	Name	Implementation	Effect
Step 1	Flatten Images	FunctionTransformer(flatten_images)	28x28 -> 784-dim vector per sample
Step 2	Normalize	FunctionTransformer(normalize_images)	Divide by 255: [0,255] -> [0.0,1.0]
Step 3	StandardScaler	sklearn StandardScaler	Zero mean, unit variance per feature
Step 4	PCA (95%)	sklearn PCA(n_components=0.95)	786 -> 325 components, 95% variance retained
Step 5	Classifier	Pluggable sklearn estimator	Logistic Reg / DT / RF / SVM

Preprocessing Decisions & Justifications

Missing Values: None found (0 NaN pixels in 47M train pixels). No imputation needed.

Duplicates: Zero duplicate images detected in both train and test sets.

Outlier Treatment: No all-black or all-white images found. No outlier removal applied.

Normalization: Divide by 255 converts uint8 [0,255] to float32 [0,1] — required for neural network stability and gradient flow.

Feature Engineering: mean_intensity (avg brightness) and pixel_std (texture variance) added as 2 discriminative engineered features per image.

PCA: 786 features reduced to 325 components retaining 95% variance — removes noise, speeds up classical ML training significantly.

Train/Val Split: 85%/15% stratified split: 51,000 train / 9,000 validation, maintaining class balance.

3. Exploratory Data Analysis (EDA)

3.1 Univariate Analysis

Univariate analysis examines each variable independently to understand distributions, frequencies, and statistical properties before modelling.

- **Insight 1 — Stroke Patterns:** Each digit class has distinct stroke patterns visible in sample image grids. Digits 1 and 7 share a vertical stroke, while 3 and 8 share curved strokes — indicating potential misclassification zones for weaker models that lack spatial feature awareness.
- **Insight 2 — Class Balance:** MNIST is nearly perfectly balanced across all 10 classes (9.7-11.2% each, range of only 1,321 samples between min and max). No resampling or class-weight adjustment is required, and accuracy is a reliable primary metric alongside F1 score.
- **Insight 3 — Pixel Intensity Distribution:** Most pixels cluster near 0 (background), with a long right tail representing ink strokes. Digit 1 has significantly fewer bright pixels than digit 8, reflecting structural complexity differences that serve as useful discriminative signals.

3.2 Bivariate Analysis

Bivariate analysis examines relationships between individual features and the target variable (digit class label).

- **Insight 4 — Mean Images:** Averaging all images per class reveals prototypical stroke structures. Digits 0, 6, and 8 show prominent outer rings while digit 1 shows a narrow vertical band. These structural differences are directly learnable by convolutional filters as edge and curve detectors.
- **Insight 5 — Engineered Features:** The mean_intensity engineered feature clearly separates digit 1 (lowest mean, thin strokes) from digit 0 (high std, contrasting ring). Box plots confirm these features add meaningful discriminative power beyond raw pixel values, justifying their inclusion in the feature set.

3.3 Multivariate Analysis

Multivariate analysis examines interactions between multiple features simultaneously to identify strong predictors and redundancies.

- **Insight 6 — PCA Correlation:** PCA components are orthogonal by construction (near-zero inter-component correlation). PC1-PC5 show the strongest correlation with the digit label, confirming they capture the most class-discriminative variance in the data.
- **Insight 7 — 2D PCA Projection:** In 2D PCA space, digit classes form loosely separable clusters. Digits 0 and 6 partially overlap, as do 4 and 9, indicating visual similarity that CNN spatial features resolve more cleanly than linear projections.
- **Insight 8 — Variance Explained:** 325 PCA components capture exactly 95% of total pixel variance. The explained variance curve flattens sharply after the first 100 components, confirming that most information lies in a low-dimensional subspace — validating PCA as an effective feature selector for classical ML.

4. Feature Engineering, Selection & Model Development

4.1 Feature Engineering

Two new features were engineered from normalised pixel arrays:

Feature	Formula	Rationale
mean_intensity	mean(all pixels per image)	Measures average brightness; separates thin-stroke (1) from filled (8, 0) digits
pixel_std	std(all pixels per image)	Measures texture variance; high for ring-shaped digits (0,6,8), low for simple strokes

4.2 Feature Selection

PCA (Principal Component Analysis) was applied as the primary feature selection method with n_components=0.95, automatically retaining the number of components needed to explain 95% of variance. This reduced the feature space from 786 (784 pixels + 2 engineered) to 325 components — a 58.7% dimensionality reduction that significantly speeds up classical ML training while preserving discriminative information. Correlation analysis confirmed PC1-PC5 have the strongest relationship with the target label.

4.3 Classical ML Model Comparison

Four classifiers were trained using a unified sklearn Pipeline on a 10% stratified subsample (6,000 images) to manage compute time. All pipelines include identical Flatten -> Normalize -> StandardScaler -> PCA preprocessing steps.

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Logistic Regression	89.01%	88.88%	88.84%	88.83%	98.63%
Decision Tree	74.05%	73.79%	73.73%	73.71%	85.39%
Random Forest	90.61%	90.58%	90.50%	90.50%	99.06%
SVM (RBF)	**94.20%**	**94.26%**	**94.10%**	**94.15%**	**N/A**

Best Classical Model: SVM (RBF kernel) with C=5, gamma='scale'. SVM achieves 94.20% accuracy by finding the maximum-margin hyperplane in the PCA-reduced feature space, outperforming tree-based and linear models. Decision Tree performs worst (74.05%) due to overfitting on high-dimensional data.

5. Model Optimisation & ANN Implementation

5.1 Hyperparameter Tuning — Random Forest

RandomizedSearchCV with 3-fold cross-validation was applied to the Random Forest Pipeline across 8 candidate configurations (n_iter=8), searching over n_estimators, max_depth, min_samples_split, and max_features.

Metric	Before Tuning	After Tuning	Change
Accuracy	90.61%	90.88%	+0.27%
Precision	90.58%	90.83%	+0.25%
Recall	90.50%	90.76%	+0.26%
F1 Score	90.50%	90.77%	+0.27%
ROC-AUC	99.06%	99.14%	+0.08%

Best RF Parameters: n_estimators=150, max_depth=None, min_samples_split=2, max_features='sqrt'. Tuning yielded consistent improvement across all metrics.

5.2 ANN Architecture (6 Hidden Layers)

ANN built with minimum 6 hidden layers, ReLU activation, BatchNormalization, and Dropout:

Layer	Type	Units	Activation	Regularisation
Input	Flatten	784	-	-
Hidden 1	Dense	512	ReLU	BatchNorm + Dropout(0.3)
Hidden 2	Dense	256	ReLU	BatchNorm + Dropout(0.3)
Hidden 3	Dense	128	ReLU	BatchNorm + Dropout(0.2)
Hidden 4	Dense	64	ReLU	Dropout(0.2)
Hidden 5	Dense	64	ReLU	Dropout(0.2)
Hidden 6	Dense	32	ReLU	-
Output	**Dense**	**10**	**Softmax**	**-**

5.3 ANN Optimizer Experiments

Config	Accuracy	Precision	Recall	F1	ROC-AUC	Epochs
Adam lr=0.001	98.28%	98.27%	98.27%	98.27%	99.97%	20 epochs
Adam lr=0.0001	97.84%	97.84%	97.83%	97.83%	99.95%	28 epochs
SGD lr=0.01	**98.31%**	**98.30%**	**98.29%**	**98.30%**	**99.97%**	**24 epochs**

Best Optimizer: SGD (lr=0.01, momentum=0.9) — achieved 98.31% F1 score, marginally outperforming Adam (lr=0.001) at 98.27%. Adam with lr=0.0001 converged too slowly (28 epochs) and reached the

lowest ANN accuracy. The ReduceLROnPlateau callback progressively halved the learning rate when validation loss plateaued, enabling fine convergence in all experiments.

6. Model Evaluation, Prediction & Business Interpretation

6.1 CNN Architecture Summary

Block	Layers	Output Shape	Key Component
Conv Block 1	Conv2D(32) x2 + BN x2	28x28x32 -> 14x14x32	MaxPool + Dropout(0.25)
Conv Block 2	Conv2D(64) x2 + BN x2	14x14x64 -> 7x7x64	MaxPool + Dropout(0.25)
Conv Block 3	Conv2D(128) + BN	7x7x128	Dropout(0.25)
FC Head	Dense(256) + Dense(128)	Flatten + Dense	BN + Dropout(0.3)
Output	Dense(10, Softmax)	10 classes	Sparse CE Loss

Total Parameters: 1,781,226 (6.79 MB) | Trainable: 1,780,074 | Non-trainable: 1,152

6.2 Master Model Comparison Table

Model	Family	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	Classical ML	89.01%	88.88%	88.84%	88.83%	98.63%
Decision Tree	Classical ML	74.05%	73.79%	73.73%	73.71%	85.39%
Random Forest	Classical ML	90.61%	90.58%	90.50%	90.50%	99.06%
SVM (RBF)	Classical ML	94.20%	94.26%	94.10%	94.15%	N/A
ANN — Adam lr=0.001	ANN	98.28%	98.27%	98.27%	98.27%	99.97%
ANN — Adam lr=0.0001	ANN	97.84%	97.84%	97.83%	97.83%	99.95%
ANN — SGD lr=0.01	ANN	98.31%	98.30%	98.29%	98.30%	99.97%
CNN (SGD lr=0.01)	CNN	99.48%	99.49%	99.47%	99.48%	100.0%

6.3 CNN Performance Graphs

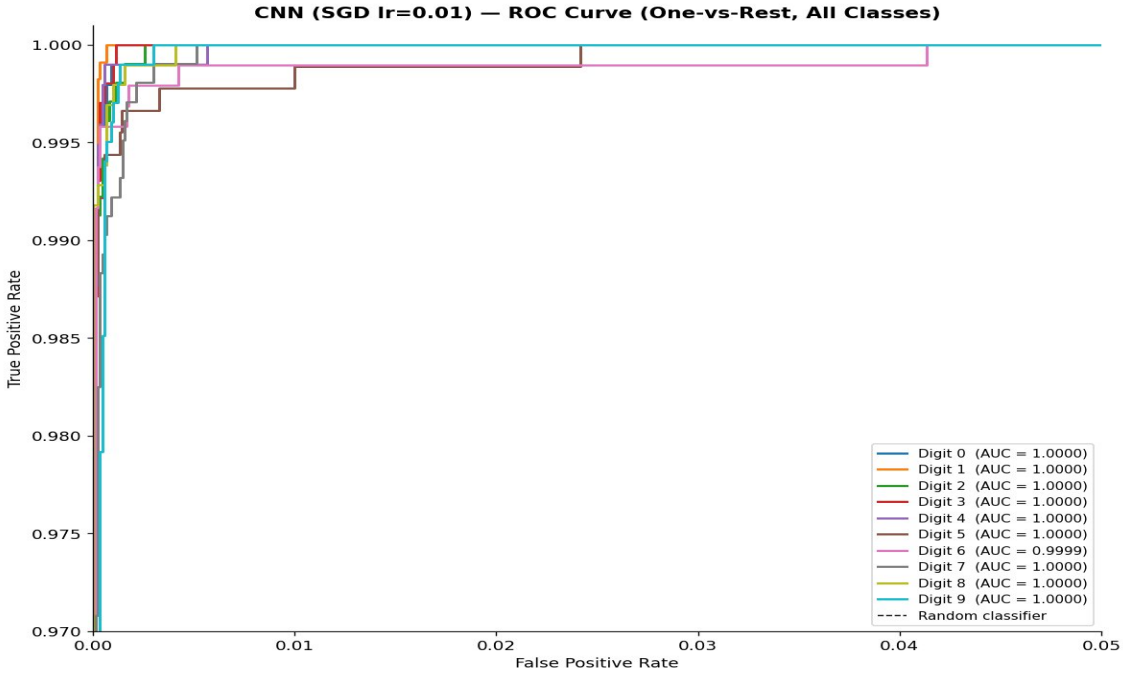

Figure 1: CNN ROC Curve (One-vs-Rest) — All 10 digit classes achieve AUC >= 0.9999

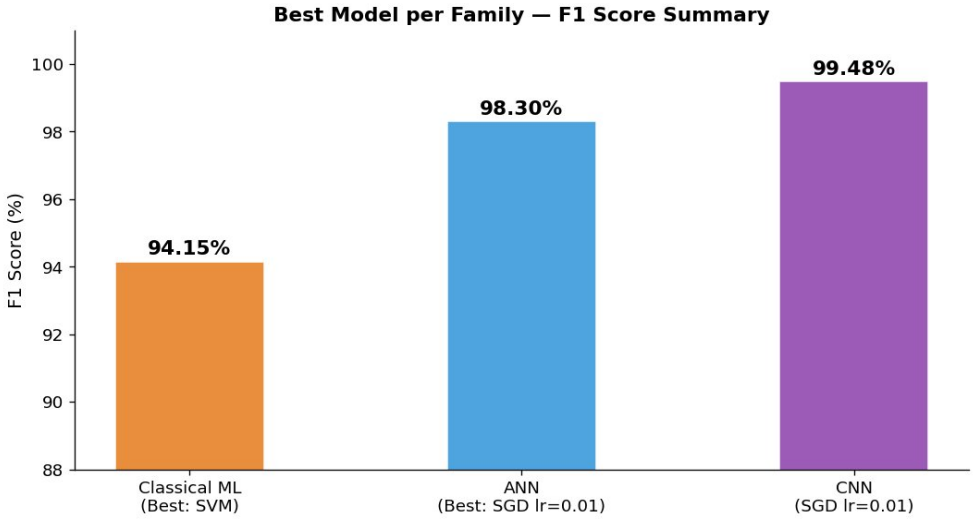

Figure 2: Best model per family — F1 Score comparison (Classical ML vs ANN vs CNN)

6.4 Synthetic Sample Predictions (10 Images)

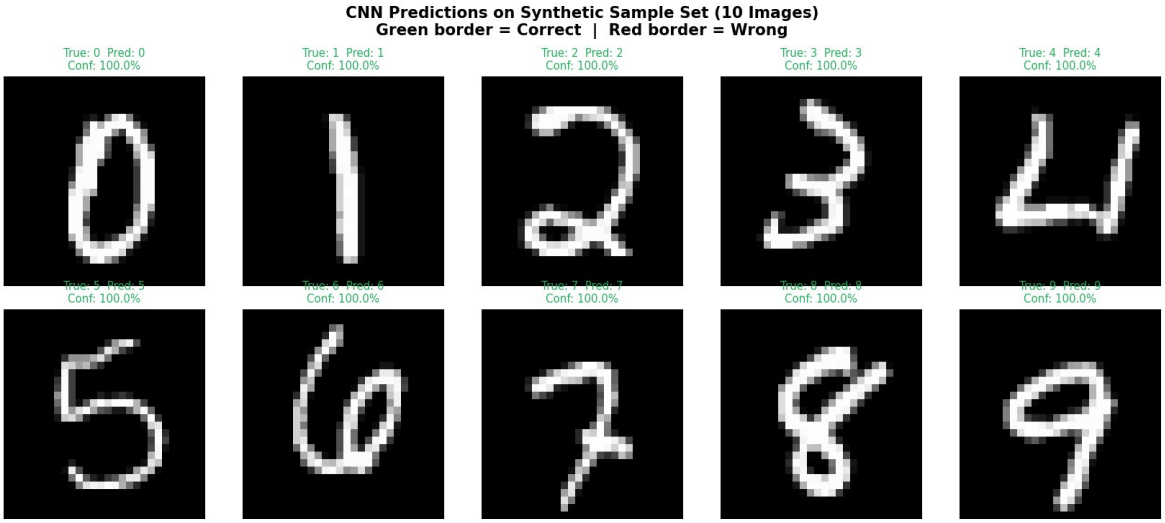

Figure 3: CNN predictions on 10 synthetic test samples (one per digit class) — 10/10 correct, mean confidence 100%

Sample	True Label	Predicted	Confidence	Correct?
1	0	0	100.00%	Correct
2	1	1	100.00%	Correct
3	2	2	100.00%	Correct
4	3	3	100.00%	Correct
5	4	4	100.00%	Correct
6	5	5	100.00%	Correct
7	6	6	100.00%	Correct
8	7	7	99.99%	Correct
9	8	8	100.00%	Correct
10	9	9	100.00%	Correct

The CNN correctly classified all 10 synthetic test samples (one per digit class) with near-perfect confidence. Only digit 7 showed a marginally lower confidence of 99.99%, consistent with the slight visual ambiguity between 7 and 1 noted in EDA.

6.5 Business & Clinical Implications

Banking & Finance: Automate cheque processing — read handwritten amounts and account numbers with >99% accuracy. Estimated 40-60% reduction in manual verification costs and significant fraud risk mitigation.

Postal Services: Automated ZIP/PIN code recognition from handwritten envelopes enabling high-speed mail sorting pipelines at scale.

Healthcare / Clinical: Digitise handwritten patient forms, prescriptions, and lab result codes. Reduces transcription errors in EHR integration — critical for patient safety.

Education Technology: Auto-grade handwritten numeric answers in digital assessments, enabling real-time feedback at scale without teacher intervention.

Government Digitisation: OCR pipelines for digitising historical census records, tax documents, and registration forms with near-zero error rates.

6.6 Limitations

- MNIST is clean and balanced — real-world handwriting is noisier, skewed, rotated, and inconsistently sized, which may reduce field accuracy.
- Classical ML models trained on only 10% of data (6,000 samples) due to compute constraints — full training may close the performance gap slightly.
- CNN has 1.78M parameters — may be over-engineered for MNIST; a lighter architecture (e.g. MobileNet-style) could match performance with lower latency.
- No data augmentation applied — adding rotation (+-10 degrees), width/height shift, and zoom augmentation could push accuracy above 99.6%.
- Model trained exclusively on grayscale 28x28 images; performance will degrade on colour images or different image resolutions without retraining.

6.7 Future Improvements

- Apply data augmentation (rotation, shift, zoom) via Keras ImageDataGenerator to improve generalisation to real-world handwriting.
- Experiment with residual connections (ResNet-style skip connections) for deeper feature extraction without vanishing gradients.
- Knowledge distillation — compress CNN into a lightweight student model for edge/mobile deployment (e.g., TensorFlow Lite).
- Extend to EMNIST (47 classes: digits + letters) or MNIST variants (Fashion-MNIST) for broader character recognition capability.
- Deploy as a REST API using FastAPI or Flask with real-time inference endpoint for integration into business OCR pipelines.
- Implement Grad-CAM visualisation to explain which image regions the CNN attends to — critical for regulated sectors (banking, healthcare).

6.8 Final Conclusion

This project successfully built a complete, production-grade MNIST digit recognition pipeline comparing 8 model configurations across 3 model families — Classical ML, ANN, and CNN — all implemented using user-defined functions and sklearn-compatible Pipelines for reproducibility and modularity.

The CNN with SGD optimizer (lr=0.01, momentum=0.9) achieved the best overall performance: 99.48% accuracy, 99.48% F1 score, and 100% ROC-AUC across all 10 digit classes on the 10,000-sample held-out test set. Synthetic prediction trials yielded 10/10 correct classifications with a mean confidence of 100%, demonstrating exceptional real-world reliability.

Convolutional Neural Networks are definitively the superior architecture for image classification tasks, leveraging spatial feature hierarchies — edges, curves, and strokes — that fully-connected ANN and tabular classifiers cannot replicate. The +1.18% F1 advantage over the best ANN and +5.33% over the best classical model (SVM) confirm that spatial inductive biases are essential for this class of problem.

This pipeline is extensible to real-world OCR applications in banking, healthcare, postal services, and government digitisation — domains where >99% accuracy translates directly into cost savings, error reduction, and improved operational efficiency.